

# Learning Typst

*A field guide, in miniature*

---

Typeset with the Chapter 22 template

# Contents

---

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Installing the tools .....</b>        | <b>3</b> |
| 1.1      | From a package manager .....             | 3        |
| 1.2      | Compiling a document .....               | 4        |
| <b>2</b> | <b>Your first document .....</b>         | <b>5</b> |
| 2.1      | The edit loop .....                      | 5        |
| 2.2      | Styling without formatting .....         | 5        |
| <b>3</b> | <b>Automating the boring parts .....</b> | <b>8</b> |
| 3.1      | Loops write your lists .....             | 8        |
| 3.2      | Functions bottle a pattern .....         | 9        |

PART I

---

# Getting started

# Installing the tools

---

Before you can typeset a single page you need the compiler on your machine. Typst ships as one self-contained binary: there are no dependencies to chase down and no runtime to install first. Download it, put it on your path, and you are ready.

## 1.1 From a package manager

The quickest route is whatever package manager you already use. On macOS with Homebrew:

```
brew install typst
```

On Windows the equivalent is `winget install --id Typst.Typst`, and most Linux distributions carry it too. Check the version once it lands:

```
typst --version
```

### NOTE

You do not have to install anything at all to try Typst. The web app at `typst.app` runs the same compiler in your browser, with a live preview beside your source. Everything in this book works identically there.

## 1.2 Compiling a document

A Typst project is just a folder of plain-text files. Point the compiler at the entry file and it writes a PDF next to it:

```
typst compile main.typ out.pdf
```

Add `--watch` instead of `compile` and it rebuilds every time you save, which is the way you will actually work once a document gets long.

### TIP

Keep the PDF open in a viewer that reloads on change. With `typst watch` running on one side and the PDF on the other, you get the same edit-and-see loop the web app gives you, without leaving your editor.

## CHAPTER 2

# Your first document

---

A blank file is already a valid Typst document. You add content with a little markup: an `=` starts a heading, asterisks make `*bold*`, underscores make `_emphasis_`, and a blank line starts a new paragraph. If you have written Markdown, none of this will surprise you.

## 2.1 The edit loop

The rhythm of working in Typst is a tight loop, shown in Figure 1. You change a line, the compiler runs in milliseconds, and the preview updates before you have looked away. That speed is not a luxury; it changes how you learn the tool, because trying something costs nothing.



Figure 1: The loop you live in: edit, compile, preview.

## 2.2 Styling without formatting

The move that takes ten minutes to get used to and pays off forever: you stop **formatting** text and start **labelling** it. Instead of selecting a line and clicking a bigger font, you mark it as a heading —

**= A section title**

The paragraph that follows it.

— and decide, once and in one place, what every heading looks like. That “one place” is a template, and building a good one is what the rest of this book is about.

**IMPORTANT**

Structure first, appearance second. A document whose headings are real headings, not just big bold text, is one Typst can build a table of contents from, cross-reference, and restyle wholesale. Fake it and you lose all three.

PART II

---

# Going further



# Automating the boring parts

---

Underneath the friendly markup sits a real programming language, and you reach it with a `#`. That is the feature that separates Typst from ordinary markup: your document can compute parts of itself.

### 3.1 Loops write your lists

Suppose you want a table of the first few powers of two. You could type it out, or you could describe it once with a loop and let the machine elaborate:

```
#for n in range(1, 6) [  
  / $2^#n$: #calc.pow(2, n)  
]
```

Change the `6` to `60` and you get sixty rows without touching the list. The same idea — a loop over some data — fills in a multiplication table, a calendar, or a hundred certificates with a hundred different names.

**WARNING**

A computed list is only as trustworthy as its input. If the data feeding a loop is wrong, the loop will cheerfully typeset the wrong thing sixty times over. Check the source, not just the page.

## 3.2 Functions bottle a pattern

When the same shape of content repeats — a callout box, a labelled figure, a styled title — you wrap it in a function and call it by name. The edit loop from Figure 1 still applies: define the function, use it, watch the preview, adjust. A book’s worth of these, gathered into files, is exactly the template this book builds in its final part.

**TIP**

The rule of thumb: the second time you copy and paste a piece of layout, stop and make it a function. The third time, you will be glad you did.

# Index

---

**C**

compiler . . . . . 3, 4

**D**

dependencies . . . . . 3

**F**

function . . . . . 9

**L**

loop . . . . . 8

**M**

markup . . . . . 5

**T**

template . . . . . 6, 9